

of configuration settings to pass to a function call. This is useful especially for functions and methods that accept a large number of keyword arguments.

By modifying a list or dictionary at runtime, you can supply a variable number of arguments for a function call using the `*` and `**` syntax.

Using * to Create Variadic Functions

You can also use the `*` syntax in `def` statements to create *variadic* or *varargs* functions that receive a varying number of positional arguments. For instance, `print()` is a variadic function, because you can pass any number of strings to it: `print('Hello!')` or `print('My name is', name)`, for example. Note that although we used the `*` syntax in function calls in the previous section, we use the `*` syntax in function definitions in this section.

Let's look at an example by creating a `product()` function that takes any number of arguments and multiplies them together:

```
>>> def product(*args):
...     result = 1
...     for num in args:
...         result *= num
...     return result
...
>>> product(3, 3)
9
>>> product(2, 1, 2, 3)
12
```

Inside the function, `args` is just a regular Python tuple containing all the positional arguments. Technically, you can name this parameter anything, as long as it begins with the star (`*`), but it's usually named `args` by convention.

Knowing when to use the `*` takes some thought. After all, the alternative to making a variadic function is to have a single parameter that accepts a list (or other iterable data type), which contains a varying number of items. This is what the built-in `sum()` function does:

```
>>> sum([2, 1, 2, 3])
8
```
